

Relatório de Auditoria de Segurança

ZapLiga / ZapCall — plataforma de discador WhatsApp multi-tenant

Data do relatório: 29 de agosto de 2026

Escopo auditado: Monorepo apps/api (NestJS + TypeScript + pg + Redis + NATS), apps/web (React + Vite), infraestrutura Docker Compose e CI/CD (GitHub Actions).

Nota metodológica

Esta auditoria cobre cinco categorias de falha comuns em SaaS multi-tenant, mapeadas para a stack real do projeto antes de qualquer análise: (1) isolamento de tenant — aqui feito por guards NestJS manuais em vez de RLS de banco, já que o projeto usa `pg` puro sem Supabase; (2) permissão definida apenas no navegador — cruzada com `@Roles`/`RolesGuard` no servidor; (3) IDOR — verificado rota a rota nos 15 controllers do backend; (4) chaves expostas — cobrindo código-fonte, Docker Compose, `.env.example`, scripts de bootstrap, CI/CD e histórico do Git; (5) XSS — verificado no frontend React e nos e-mails HTML gerados pelo backend. Cada achado reportado foi lido e confirmado diretamente no código-fonte; nenhum item é especulativo.

Backend. NestJS 11 / TypeScript, acesso a dados via SQL parametrizado cru com o driver `pg` (sem ORM), autenticação JWT (access token curto + refresh token rotativo em cookie httpOnly).

Isolamento multi-tenant. Não há RLS (não é Supabase/Postgres RLS) nem middleware automático de tenant. O isolamento é feito manualmente por um par de guards NestJS (`AuthGuard` + `TenantMembershipGuard`) que resolve `request.tenantId` a partir da rota/cabeçalho `x-tenant-id` e valida contra `tenant\_memberships`; cada consulta SQL then precisa filtrar explicitamente por `tenant\_id`. A auditoria de item 1 mapeou esse mecanismo e verificou, controller a controller e serviço a serviço, se o filtro é aplicado.

Permissão/autorização. RBAC via decorator `@Roles(...)` + `RolesGuard`, aplicado no servidor em toda rota; o item 2 cruzou cada gate visual do frontend (React) com o guard correspondente no backend.

IDOR. Item 3 percorreu todos os 15 controllers e os serviços por trás deles, checando se toda busca/alteração/remoção por `:id` também filtra por `tenant\_id` (ou por posse explícita, nos casos sem tenant, como visualizações salvas).

Segredos. Item 4 cobriu código-fonte, ``docker-compose.yml``, ``env.example``, scripts de bootstrap, workflow de CI/CD e histórico do Git.

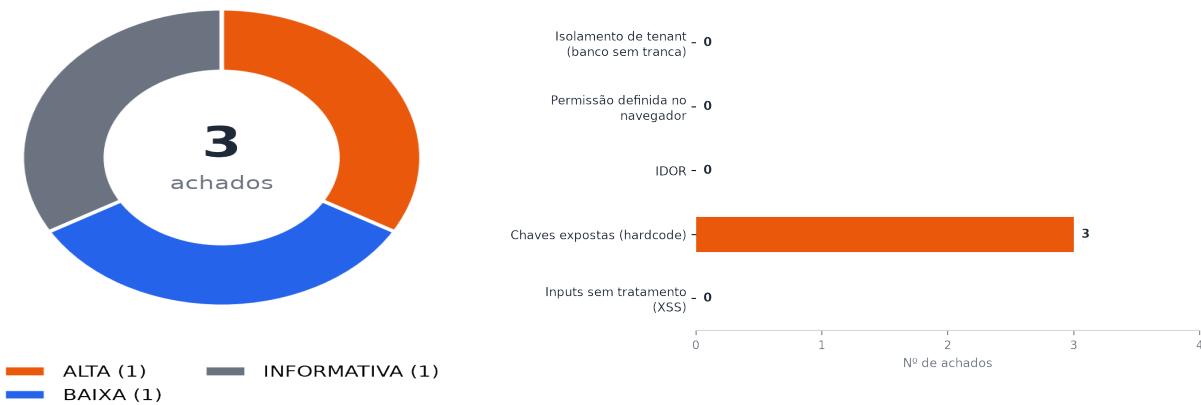
Frontend / XSS. React + Vite, sem framework SSR. Item 5 verificou uso de ``dangerouslySetInnerHTML`/`innerHTML`/`eval``, URLs controladas por usuário e, no backend, geração de e-mails HTML.

Resumo executivo

3 achados confirmados no código, nenhum classificado como crítico. ALTA: 1 · BAIXA: 1 · INFORMATIVA: 1.

Os itens 1 (isolamento de tenant), 2 (permissão no navegador), 3 (IDOR) e 5 (XSS) não apresentaram achados exploráveis após revisão completa arquivo por arquivo — ver seção de pontos fortes para a evidência detalhada de cobertura. Os três achados do item 4 (chaves expostas) dizem respeito a como os segredos padrão de desenvolvimento se comportam em uma implantação de produção mal configurada.

Achados por severidade



Distribuição por categoria auditada

Categoria	Resultado
1. Isolamento de tenant (banco sem tranca)	Sem achados
2. Permissão definida no navegador	Sem achados
3. IDOR	Sem achados
4. Chaves expostas (hardcode)	3 achados (F1, F2, F3)
5. Inputs sem tratamento (XSS)	Sem achados

Pontos fortes (verificados no código)

- **Isolamento multi-tenant consistente.** Os 15 controllers do backend (``admin``, ``tenants``, ``users``, ``memberships``, ``leads``, ``lead-folders``, ``numbers``, ``sdrs``, ``calls``, ``dialer``, ``invitations``, ``metrics``, ``audit``) foram lidos por completo. Todas as rotas que expõem dados de um tenant usam ``AuthGuard`` + ``TenantMembershipGuard`` (`auth.guards.ts:40-80`), que resolve ``tenantId`` a partir da rota/cabeçalho e valida contra ``tenant_memberships`` antes de popular ``request.tenantId``. Toda consulta SQL revisada nos serviços (``leads.controller.ts``, ``lead-folders.service.ts``, ``numbers.controller.ts``, ``dialer.service.ts`` — 990 linhas —, ``metrics.repository.ts`` — 516 linhas —, entre outros) inclui ``WHERE tenant_id = $1`` (ou join equivalente por ``tenant_id``) antes de filtrar por qualquer ``id`` vindo do cliente.
- **RBAC reforçado no servidor, não só na UI.** O frontend (``App.tsx``, ``AccessPage.tsx``, ``AdminPage.tsx``) esconde telas administrativas com flags como ``isSdr``, ``isSuperAdmin``, ``canManageNumbers``, mas cada ação correspondente tem um ``@Roles(...)`` no controller do NestJS, verificado pelo ``RolesGuard`` (`auth.guards.ts:82-95`): ``AdminController`` e ``TenantsController`` (rotas de criação/limite) exigem ``@Roles('super_admin')``; ``MembershipsController`` restringe promoção a líder e mudança de papel a ``super_admin``; ``NumbersController``, ``SdrsController``, ``LeadsController`` e ``CallsController`` exigem ``leader`/`super_admin`` com exceções pontuais e explícitas para ``sdr`` (ex.: ``SdrsController.own()``, ``finishPause()`` com checagem extra de posse em `memberships.controller.ts:34-42` e `sdrs.controller.ts:66-69`). Chamar a API diretamente sem passar pela UI não contorna nenhuma dessas permissões.
- **Nenhum IDOR sobrevivendo à revisão.** Todas as rotas que recebem ``:id`` (leads, pastas, números, SDRs, chamadas, convites, metas de métricas, visualizações salvas, exportações) foram conferidas: o ``id`` do recurso é sempre combinado com ``tenant_id = $1 AND id = $2`` na query (ex.: ``numbers.controller.ts:94-98``, ``lead-folders.service.ts:32-47``, ``metrics-export.repository.ts:43-54``). Nos dois casos sem tenant (visualizações salvas em ``metrics-views.service.ts``), a posse é checada explicitamente por ``owner_user_id !== userId`` antes de editar/remover.
- **SQL parametrizado de ponta a ponta.** Não há ORM, mas o padrão do projeto é disciplinado: nomes de coluna interpolados nas queries vêm sempre de listas fixas no código (nunca de entrada do usuário) e todo valor de filtro é passado como parâmetro posicional (``$1``, ``$2``, ``ANY($n::text[])``), inclusive em filtros dinâmicos de métricas com múltiplos arrays (``metrics.repository.ts:380-460``) e na busca textual da auditoria (``audit.service.ts:35``, usa ``ILIKE $n`` com o termo passado como parâmetro, não concatenado).
- **Segurança de autenticação e sessão.** Login com bloqueio progressivo por tentativa (``auth.service.ts:26-37``), hashes bcrypt com custo configurável, refresh tokens rotativos com detecção de reuso e revogação em cascata da família de sessão (``auth.service.ts:104-125``), tickets de WebSocket de uso único e curta duração (``auth.service.ts:159-183``), e validação de posse do SDR antes de anexar o socket de mídia (``sdr.gateway.ts:133-138``).
- **Nenhum segredo real commitado.** Buscas em todo o código-fonte e no histórico completo do Git (28 commits) não encontraram nenhuma chave de API, senha ou token real; apenas o ``.env.example`` com valores de exemplo foi versionado. ``.gitignore`` exclui ``.env`` e ``backups/``, e o workflow de deploy exclui ``.env`` do pacote enviado ao servidor (``deploy.yml:80``).
- **Mass assignment bloqueado globalmente.** ``main.ts:26`` registra ``ValidationPipe({ whitelist: true, forbidNonWhitelisted: true })`` de forma global, e todo DTO revisado usa ``class-validator`` com listas fechadas (``@IsIn(['leader','sdr'])`` em ``update-membership-role.dto.ts``, ``@IsIn(['user','super_admin'])`` em ``update-platform-role.dto.ts``), impedindo que um cliente injete papéis ou campos não previstos.
- **Nenhum vetor de XSS encontrado no frontend.** Busca completa em ``apps/web/src`` (44 arquivos) por ``dangerouslySetInnerHTML``, ``innerHTML``, ``outerHTML``, ``eval`` e ``new Function`` não retornou nenhuma ocorrência; toda renderização passa pelo escape automático do React. Não há HTML/Markdown de usuário

renderizado sem sanitização.

- **E-mails HTML escapados no backend.** ``invitation-mailer.ts:6-11,31-39`` escapa manualmente ``tenantName``, ``role`` e a URL do convite antes de interpolá-los no corpo HTML do e-mail — trata corretamente a única superfície de geração de HTML a partir de dados controlados por usuário (nome da empresa) encontrada no backend.

- **Token de acesso fora do `localStorage`.** ``apps/web/src/services/api.ts:2-16`` mantém o access token apenas em memória (variável de módulo) e sincroniza entre abas via ``BroadcastChannel``, evitando persistência em ``localStorage`/`sessionStorage`` — reduz a superfície de roubo de token mesmo na ausência de um XSS conhecido.

Pontos fracos (riscos centrais)

O único risco central identificado é operacional, não de código: a cadeia de defaults de ambiente (F1) pode desligar silenciosamente as proteções de produção caso o operador não sobrescreva ``NODE_ENV``. Os achados F2 e F3 são camadas de defesa em profundidade ausentes, hoje mitigadas pelo bind de rede em localhost.

Achados detalhados

Sev.	Arquivo:linha	Descrição
ALTA	.env.example:1 docker-compose.yml:97 apps/api/src/main.ts:15 apps/api/src/modules/auth/auth.service.ts:214	F1 — NODE_ENV padrão "development" desativa toda a validação de segredos e as proteções de produção O template de ambiente e o docker-compose.yml só ligam a validação de segredos, o redirecionamento HTTPS obrigatório e o cookie de refresh `Secure` quando `NODE_ENV === 'production'` (main.ts:15-22 e auth.service.ts:214). Só que o próprio template deixa `NODE_ENV` sem valor forte: `.env.example:1` traz `NODE_ENV=development`, e `docker-compose.yml:97` usa `NODE_ENV: \${NODE_ENV:-development}` — ou seja, se o operador criar o `.env` de produção copiando o exemplo (o fluxo descrito no próprio `.github/workflows/deploy.yml`, que apenas exige "crie /opt/zapliga/.env antes do primeiro deploy") e esquecer de sobrescrever `NODE_ENV=production`, a API roda em produção real com todas as proteções de produção desligadas.
BAIXA	docker-compose.yml:6 docker-compose.yml:30 apps/api/src/main.ts:16	F2 — Credencial padrão do PostgreSQL não faz parte da validação de segredos de produção `docker-compose.yml:6-8` e `:30-32` definem `POSTGRES_USER`/`POSTGRES_PASSWORD` com fallback `zapcall`/`zapcall` (`\${POSTGRES_USER:-zapcall}`, `\${POSTGRES_PASSWORD:-zapcall}`). Diferente de `JWT_ACCESS_SECRET`/`WAXUM_API_KEY`/`WAXUM_JWT_SECRET`, esse par não está no conjunto `unsafeDefaults` verificado em `main.ts:16` — não existe nenhuma checagem de startup que rejeite a senha de exemplo do banco em produção.
INFORMATIVA	docker-compose.yml:40	F3 — Redis roda sem autenticação (sem `requirepass`) O serviço `redis` em `docker-compose.yml:40-51` não define `requirepass`/`--requirepass`, então qualquer processo capaz de alcançar `127.0.0.1:6379` tem acesso irrestrito — inclusive às chaves de reserva de chamadas e ao cache de LID do WhatsApp (dialer.service.ts) usados pelo discador.

ALTA

F1 — NODE_ENV padrão "development" desativa toda a validação de segredos e as proteções de produção

Categoria: Chaves expostas (hardcode)

Local: .env.example:1 · docker-compose.yml:97 · apps/api/src/main.ts:15 · apps/api/src/modules/auth/auth.service.ts:214

Descrição: O template de ambiente e o docker-compose.yml só ligam a validação de segredos, o redirecionamento HTTPS obrigatório e o cookie de refresh `Secure` quando `NODE\_ENV === 'production'` (main.ts:15-22 e auth.service.ts:214). Só que o próprio template deixa `NODE\_ENV` sem valor forte: `.env.example:1` traz `NODE\_ENV=development`, e `docker-compose.yml:97` usa `NODE\_ENV: \${NODE\_ENV:-development}` — ou seja, se o operador criar o `.env` de produção copiando o exemplo (o fluxo descrito no próprio `.github/workflows/deploy.yml`, que apenas exige "crie /opt/zapliga/.env antes do primeiro deploy") e esquecer de sobrescrever `NODE\_ENV=production`, a API roda em produção real com todas as proteções de produção desligadas.

Por que é explorável: Com `NODE\_ENV` diferente de `production`, o bloco de main.ts:15-22 nunca executa — logo os segredos padrão públicos (visíveis neste próprio repositório) `JWT\_ACCESS\_SECRET=change-this-access-secret`, `WAXUM\_API\_KEY=zapcall-local-waxum-token` e `WAXUM\_JWT\_SECRET=change-this-local-secret` continuam válidos e **sem checagem alguma que rejeite o valor de exemplo**. Qualquer pessoa que conheça esses defaults (públicos no histórico do projeto) pode assinar um JWT de acesso válido com `jsonwebtoken` usando o segredo padrão, definindo `sub` como o UUID de qualquer usuário existente — `AuthGuard` (auth.guards.ts:30-35) apenas verifica a assinatura e busca o usuário por esse `sub`, sem nenhum vínculo adicional ao segredo real do ambiente. O resultado é personificação completa de qualquer conta, incluindo um `super\_admin` cujo UUID tenha vazado por qualquer canal (ex.: listagem de membros de um tenant, logs, etc.). Adicionalmente, o cookie de refresh deixa de ser `Secure` (auth.service.ts:214), podendo trafegar em texto claro se a borda TLS não repassar `x-forwarded-proto` corretamente.

Condição de explorabilidade: Requer que o operador não sobrescreva `NODE\_ENV=production` no `.env` real do servidor — um erro plausível porque o próprio arquivo de exemplo do projeto já vem com `NODE\_ENV=development`. Não requer nenhuma outra falha de configuração.

BAIXA

F2 — Credencial padrão do PostgreSQL não faz parte da validação de segredos de produção

Categoria: Chaves expostas (hardcode)

Local: docker-compose.yml:6 · docker-compose.yml:30 · apps/api/src/main.ts:16

Descrição: `docker-compose.yml:6-8` e `:30-32` definem `POSTGRES\_USER`/`POSTGRES\_PASSWORD` com fallback `zapcall`/`zapcall` (`\${POSTGRES\_USER:-zapcall}`, `\${POSTGRES\_PASSWORD:-zapcall}`). Diferente de `JWT\_ACCESS\_SECRET`/`WAXUM\_API\_KEY`/`WAXUM\_JWT\_SECRET`, esse par não está no conjunto `unsafeDefaults` verificado em `main.ts:16` — não existe nenhuma checagem de startup que rejeite a senha de exemplo do banco em produção.

Por que é explorável: Hoje o risco é mitigado porque `docker-compose.yml:10` publica a porta do Postgres apenas em `127.0.0.1:5432`, não acessível externamente por padrão. O achado é a ausência de defesa em profundidade: uma mudança futura no compose (ex.: trocar o bind para `0.0.0.0` para depuração remota) ou uma rede Docker mal configurada exporia um banco protegido apenas por `zapcall:zapcall`, com acesso completo a dados de todos os tenants.

Condição de explorabilidade: Só é explorável se a porta 5432 for exposta além de `127.0.0.1` (configuração adicional além do padrão do repositório).

INFORMATIVA

F3 — Redis roda sem autenticação (sem `requirepass`)

Categoria: Chaves expostas (hardcode)

Local: docker-compose.yml:40

Descrição: O serviço `redis` em `docker-compose.yml:40-51` não define `requirepass`/`--requirepass`, então qualquer processo capaz de alcançar `127.0.0.1:6379` tem acesso irrestrito — inclusive às chaves de reserva de chamadas e ao cache de LID do WhatsApp (dialer.service.ts) usados pelo discador.

Por que é explorável: Mitigado da mesma forma que o achado F2: a porta só é publicada em `127.0.0.1:6379`. Registrado para reforçar a defesa em profundidade, sobretudo se o Redis um dia precisar ser compartilhado entre hosts.

Condição de explorabilidade: Só é explorável se a porta 6379 for exposta além de localhost.

Recomendações priorizadas

Prior.	Ação	Justificativa
P1	Tornar a validação de segredos de produção independente de `NODE_ENV`	Corrige F1: mover a checagem de `unsafeDefaults` em `main.ts:15-22` para rodar sempre (ou definir `NODE_ENV=production` como padrão explícito no `docker-compose.yml` de implantação, exigindo opt-in para `development`). Aplicar o mesmo padrão ao `AUTH_COOKIE_SECURE` em `auth.service.ts:214`, para que o cookie de refresh só saia sem `Secure` com uma flag explícita, nunca por omissão de `NODE_ENV`.
P2	Incluir a credencial do PostgreSQL na validação de segredos e evitar expor a porta além de localhost	Corrige F2: adicionar `POSTGRES_PASSWORD` (e o mesmo para `SUPERADMIN_PASSWORD`, já checado apenas no script `bootstrap-admin.ts`) ao conjunto de defaults rejeitados em produção, e manter o bind de rede restrito a `127.0.0.1` em qualquer mudança futura no `docker-compose.yml`.
P3	Habilitar autenticação no Redis (`requirepass`)	Corrige F3: definir uma senha para o Redis via variável de ambiente e propagá-la para `REDIS_URL` nos serviços que o consomem, mantendo o bind em localhost como camada adicional.
P4	Formalizar este relatório como checklist de release	Nenhuma correção de código é necessária para os itens 1, 2, 3 e 5 desta auditoria — o recomendado é adicionar uma checagem automatizada (ex.: `scripts/verify-multitenant.mjs`, já existente no projeto) que rode em CI e falhe se uma nova rota for adicionada sem `TenantMembershipGuard`/`RolesGuard`, preservando a consistência encontrada nesta revisão.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de uma issue. Achados triviais do mesmo tema (F2 e F3, ambos sobre credenciais/portas de infraestrutura) foram agrupados em uma única issue para evitar spam.

--- ISSUE 1 ---

[Segurança] NODE_ENV padrão "development" desativa a validação de segredos e o hardening de produção

Labels sugeridas: security, alta

Problema

apps/api/src/main.ts:15-22 só valida se JWT_ACCESS_SECRET, WAXUM_API_KEY e WAXUM_JWT_SECRET ainda estão com o valor de exemplo, força HTTPS e ativa trust proxy apenas quando process.env.NODE_ENV === 'production'. O mesmo vale para o cookie de refresh ser Secure (apps/api/src/modules/auth/auth.service.ts:214).

Só que .env.example:1 traz NODE_ENV=development e docker-compose.yml:97 usa NODE_ENV: \${NODE_ENV:-development} como fallback. O fluxo de deploy documentado em .github/workflows/deploy.yml só pede para o operador criar um .env em /opt/zapliga/.env – se esse arquivo for baseado no exemplo e não sobrescrever NODE_ENV, a API de produção roda com o hardening inteiro desligado.

Por que é explorável

Sem o bloco de main.ts:15-22 rodando, os segredos padrão (change-this-access-secret, zapcall-local-waxum-token, change-this-local-secret) continuam ativos sem nenhuma checagem. Esses valores são públicos (estão neste repositório). Quem os conhece pode assinar um JWT válido com jsonwebtoken, definindo sub como o UUID de qualquer usuário – AuthGuard (apps/api/src/modules/auth/auth.guards.ts:22-38) apenas verifica a assinatura e busca o usuário por esse sub, concedendo a sessão completa daquela conta (inclusive super_admin, se o UUID for conhecido).

Evidência

```
// .env.example:1
NODE_ENV=development

// docker-compose.yml:97
NODE_ENV: ${NODE_ENV:-development}

// apps/api/src/main.ts:15-17
if (process.env.NODE_ENV === 'production') {
  const unsafeDefaults = new Set([...]);
  if (!process.env.JWT_ACCESS_SECRET || ... ) throw new Error(...);
  ...
}
```

Impacto

Personificação completa de qualquer usuário (incluindo administradores da plataforma) caso a implantação de produção não defina explicitamente NODE_ENV=production. Também expõe o cookie de refresh sem a flag Secure.

Sugestão de correção

- Executar a validação de unsafeDefaults sempre, independentemente de NODE_ENV (ela só falha se o valor realmente for um dos defaults conhecidos – não deveria depender do ambiente).
- Trocar o fallback do docker-compose.yml de produção para NODE_ENV: \${NODE_ENV:?defina NODE_ENV} ou production, exigindo opt-in explícito para rodar em modo development.
- Aplicar a mesma independência de NODE_ENV ao cálculo de cookieSecure em auth.service.ts.

Critérios de aceite

- A API recusa subir (ou loga erro crítico) se algum dos segredos estiver com valor de exemplo, mesmo com NODE_ENV não definido como production.
- O cookie de refresh só sai sem Secure mediante flag explícita, nunca por omissão de NODE_ENV.
- docker-compose.yml de produção não sobe com NODE_ENV implícito.
- Teste automatizado cobrindo a rejeição dos defaults independente de NODE_ENV.

--- FIM ISSUE 1 ---**--- ISSUE 2 ---****[Segurança] Defaults de credenciais de infraestrutura (Postgres, Redis) sem validação de startup nem autenticação**

Labels sugeridas: security, baixa

Problema

Dois pontos relacionados de "default público vira segredo real se não for sobrescrito":

1. docker-compose.yml:6-8 e :30-32 definem POSTGRES_USER/POSTGRES_PASSWORD com fallback zapcall/zapcall. Diferente de JWT_ACCESS_SECRET e WAXUM_*, essa credencial não está no conjunto unsafeDefaults validado em apps/api/src/main.ts:16.
2. O serviço redis em docker-compose.yml:40-51 não define requirepass – qualquer processo que alcance a porta tem acesso total, sem senha alguma.

Por que é explorável

Hoje ambos os serviços publicam a porta apenas em 127.0.0.1 (docker-compose.yml:10 e :43-44), então não há exposição externa por padrão. O risco é a ausência de defesa em profundidade: uma mudança futura no bind de rede, ou uma rede Docker mal configurada, exporia o banco (com a senha de exemplo zapcall) ou o Redis (sem senha nenhuma) a qualquer host que alcance a máquina.

Evidência

```
# docker-compose.yml:6-10
POSTGRES_DB: ${POSTGRES_DB:-zapcall}
POSTGRES_USER: ${POSTGRES_USER:-zapcall}
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-zapcall}
ports:
- "127.0.0.1:5432:5432"
```

```
# docker-compose.yml:40-44 (sem requirepass)
redis:
image: redis:7-alpine
ports:
- "127.0.0.1:6379:6379"
```

Impacto

Acesso total ao banco de dados (todos os tenants) ou ao Redis (reservas de chamada, cache de LID do WhatsApp) caso a porta seja exposta além de localhost em qualquer momento futuro.

Sugestão de correção

- Adicionar POSTGRES_PASSWORD (e SUPERADMIN_PASSWORD) ao conjunto unsafeDefaults validado em produção.
- Definir requirepass no Redis via variável de ambiente e propagar a senha em REDIS_URL.
- Manter o bind em 127.0.0.1 documentado como requisito de segurança, não apenas como configuração padrão incidental.

Critérios de aceite

- Subir a API em modo produção com POSTGRES_PASSWORD de exemplo falha no boot com mensagem clara.
- Redis exige autenticação (AUTH) para qualquer comando.
- Documentação do deploy menciona explicitamente por que as portas de Postgres/Redis não devem ser expostas além de localhost.

--- FIM ISSUE 2 ---